The One-Path Error: The Economics of Autonomous Coding

David Charlot, Open Interface Engineering

Economics essay · OpenIE research.openie.dev
https://research.openie.dev/papers/rce-economics/ · PDF https://research.openie.dev/pdfs/rce-economics.pdf

> The race to own the projected trillion-dollar AI coding industry assumes one path to autonomous computer automation: high energy, high compute, high entropy and disorder. That assumption is an economic error. A second path converges on code that already exists and pays for generation once per new case. Its marginal cost per covered task falls toward the energy floor. This is the economics companion to the Reality Code Engine (RCE) research paper.

The economics companion to The Reality Code Engine: One Form, Many Fits, and the Low-Entropy Path to Autonomous Coding. The engineering, the theory and the measured results live there. This essay holds the market and pricing argument.

Abstract

The pursuit to own the projected trillion-dollar AI coding industry misses. It assumes there is only one path to autonomous computer automation: high energy, high compute, high entropy and disorder. On that path every task pays for generation again, so the cost of task 10,000 equals the cost of task 1, and whoever owns the most compute owns the market.

There is a second path. People have already coded most of what will ever be coded. A program is a database functional state machine: one Form, many fits. Human intent is high entropy, the machine floor is low entropy, and code converges by design. On this path a system retrieves, adapts and proves a fit that exists, and pays a model once per new case. The RCE research paper demonstrates it: 20 of 20 tasks on a frozen repair corpus with 0 model calls.

The two paths have different cost curves. The first prices autonomy by scarcity of compute. The second drives the marginal cost of a covered task toward the energy to run it. That is the economic error in the one-path assumption: it values a market at the price of a path that the second path makes unnecessary.

Shared thesis, stated once for every paper. Energy to run is the only true metric of computer intelligence. All other factors collapse to zero. The gate form defines what that means. Among runs that pass the task's completeness predicate $C(z)$ within the stated budget and deadline, systems are ranked by joules to completion, J^*. Correctness, deadlines and task size are the gate, not terms in the score. Parameters, tokens, benchmark scores and prices carry zero weight in the ranking (Agency, Section 1). A price trend is a separate claim, labeled as one. Epoch AI reports that the cost of a given level of AI performance has fallen about 47 percent per quarter

since 2023, about 13 times per year (Emberson and Roodman, *The plunging price of thought*, 22 September 2026). If that trend continues, energy becomes the dominant marginal cost of inference. That is a cited trend with a condition, not a measured collapse. Spell Check is Global is an existence proof that some computer intelligence commoditizes to bundled zero price. Estimates are never `measured_j`.

---

1. The projection and what it assumes

The trillion-dollar figure is a value estimate, not a revenue forecast. Appenzeller and Li count about 30 million software developers, assume $100,000 of economic value per developer per year, and arrive at about 3 trillion per year of developer value that AI coding could raise [1]. Revenue estimates for AI coding tools are far smaller: about $9.35 billion in 2026 [2].

To capture value of that size, the one-path plan sells generation. The price is per token or per seat, and the cost underneath it is compute: every change is a fresh sample from a model, on every task. Three properties follow.

1. **Constant marginal cost per task.** Each task pays for generation again. Learning across tasks lowers the price per token, not the tokens per task.
2. **Scale as moat.** If every task needs frontier compute, the firm with the most compute sets the price.
3. **Waste is billed.** Spend past the point of correctness is still revenue.

---

2. The one-path assumption as an economic error

The evidence that most generation spend is waste.

- Agentic coding uses about 1,000 times more tokens than code chat, runs of the same task differ by up to 30 times in tokens, and accuracy often peaks at intermediate cost and saturates [3].
- On the SWE-bench Verified bash-only board, one closed-weight entry resolves 76.8% at $0.754 per task, and an open-weight entry resolves 75.80.073 per task [4]. One point of score costs about ten times the price.
- An output token costs 30 to 1,000 times the energy of an input or cached token [5].

Mixture of Limits names the pattern: there is a minimal amount of anything to accomplish a task, and any more is waste (Metered Commit Gates, Part II). A market priced on spend past the floor is priced on waste.

The evidence that most code already exists. Seventy percent of the files on GitHub are clones of earlier files [6]. Line-level code is near-totally redundant against a 420-million-line corpus [7]. Near-deduplication removes more than half of the largest open code corpora [8]. A catalog of fix

patterns repairs 43 real Java bugs with no learning [9]. Most of the supply the one-path plan sells as generation is retrieval.

The cost curve of the second path. Promotion pays a model once per new case. Every later case of the same shape runs on a deterministic handler at the handler floor. The cost per task then falls with coverage, not with the price per token. Compiled code generation reports break-even against runtime inference at about 17 transactions and 57 times fewer tokens at 1,000 transactions [10].

	One path: generate	**Second path: converge**
Marginal cost per task	Constant: generation on every task	Falls with coverage toward the run floor
Moat	Compute scale	None needed: open retrieval, rules and solvers
Price trend	Falls per token	Falls per token and per task
What is billed past correctness	Waste	Nothing: the gate closes the task

---

3. Two declines multiply

The market lowers the price per call. Epoch AI reports about 47 percent per quarter, about 13 times per year, since 2023 for a fixed level of performance (Emberson and Roodman, *The plunging price of thought*, 22 September 2026). That is a cited trend, labeled as one. The second path lowers the calls per task, to zero for every covered case. The cost per task is the product of the two. They are independent levers, and the one-path plan pulls only the first.

---

4. Pricing once the floor is reached

When the marginal cost of a covered coding task approaches the energy to run it, generation cannot be sold at a premium on those tasks. Spell Check is Global records the finished case: a capability whose marginal cost fell to about zero and that complemented a tool everyone already had was bundled, not sold. Covered coding tasks follow the same mechanism: their marginal cost falls to the handler floor, and they complement the editors and repositories people already have.

What remains priced is the residual: the new case no stored fit covers, and the proof that a fit is correct. Energy to run is what remains on every task.

---

5. Demand

Cheaper tasks can raise total demand. Task Satiation vs. Aggregate Rebound in AI Demand separates the two effects: per-task appetite stops at the completion predicate, and aggregate use can still grow because cheaper synthesis opens new tasks. The second path changes the

first effect, not the second. A larger market of covered tasks is a market whose marginal cost is the floor, not the frontier.

---

6. Propositions and protocols

Proposition E1 (the cost curves cross). For a stream of tasks in one repository, the cumulative cost per resolved task of the second path falls below the first after a finite number of tasks. *Protocol:* run the RCE Track B protocol and a plain generation agent on the same issue stream under a wall meter, report `measured_j` and dollars per resolved task by task index, and report the crossing index.

Proposition E2 (price follows the floor). Where a coding capability is covered by retrieval and rules, its market price falls toward the energy to run it plus the cost of the proof. *Protocol:* track list prices of a covered capability class against its measured run energy over time, in the manner of the Spell Check price series.

Proposition E3 (compute scale is not a moat on covered tasks). On tasks the second path covers, the resolve rate does not rise with the compute of the last gear. *Protocol:* run the SWE-bench Lite offline protocol with no model, a small local model and a frontier model as the last gear, and compare resolve rates on the tasks closed before the last gear.

References

[1] G. Appenzeller and Y. Li, "The Trillion Dollar AI Software Development Stack," 2025. https://a16z.com/the-trillion-dollar-ai-software-development-stack/

[2] Mordor Intelligence, "AI Code Tools Market Size, Share and 2031 Growth Trends Report," fetched 11 October 2026. https://www.mordorintelligence.com/industry-reports/artificial-intelligence-code-tools-market

[3] Bai et al., token use of eight frontier models on SWE-bench Verified, 2026. arXiv:2604.22750

[4] SWE-bench leaderboard data, bash-only view, entries dated 17 February 2026. https://github.com/SWE-bench/swe-bench.github.io

[5] Cho et al., "Librarian," 2026. arXiv:2605.27787

[6] C. V. Lopes et al., "DéjàVu: A Map of Code Duplicates on GitHub," OOPSLA 2017. doi:10.1145/3133908

[7] M. Gabel and Z. Su, "A Study of the Uniqueness of Source Code," FSE 2010. doi:10.1145/1882291.1882315

[8] The Stack v2 dataset card. https://huggingface.co/datasets/bigcode/the-stack-v2

[9] K. Liu et al., “TBar: Revisiting Template-based Automated Program Repair,” ISSTA 2019. arXiv:1903.08409

[10] Trooskens et al., “Compiled AI: Deterministic Code Generation for LLM-Based Workflow Automation,” 2026. arXiv:2604.05150